



Bahria University, Islamabad

Department of Software Engineering

Object Oriented Programming Lab (Spring-2026)

Instructor: Engr. Muhammad Faisal Zia

Group Members:

Ayan Ahmed – 09-131252-009

M Taha Kashif – 09-131252-063

Project Submission

Date: 15 May 2026

Project Details:

Project Title: Voltly (A Smart Electricity Billing & Management System)

GitHub Repository Link: <https://github.com/tahakashiff/Voltly>

Problem Statement:

Modern utility companies and households require efficient, automated, and transparent electricity billing. Manual calculations are error-prone and hard to audit. There is also a growing demand for solar net-metering support and peak-hour billing. This project addresses these needs through a C++/Qt application that automates billing, manages user/admin roles, tracks consumption/billing history, and supports solar energy credits.

Proposed Features

- **Admin Dashboard:**
 - Add, update, search, and display consumers.
 - Set and update electricity rates (peak, off-peak, commercial, solar).
 - Register/search users and view statistics.
- **User Dashboard:**
 - View linked consumer bill.
 - View usage/bill history.
 - Pay monthly bills.
- **Multi-window GUI in Qt:**
 - Separate login, admin, and user panels with modern UI.
- **Solar Net-Metering:**
 - Billing logic supports exported solar units and credits.
- **Text-file data storage:**
 - Data is persisted in plain text files for users, consumers, and bills.

OOP Design Overview

Major Classes and Responsibilities

SystemContext.cpp

- **Role:** Core business logic hub. Handles user and consumer management, data validation, billing and rate management, and acts as the bridge between the UI and File layers.
- **Examples of Responsibility:** Validates logins, interprets admin/user commands, and calls appropriate file and consumer class logic.

LoginWindow, AdminDashboard, UserDashboard (Qt)

- **Role:** Represent and manage different UI windows.
 - **LoginWindow:** Authenticates users/admins and launches dashboards.
 - **AdminDashboard:** Allows admin to manage consumers, rates, and users.
 - **UserDashboard:** Provides users with ways to view, pay bills, and see usage.

Consumer.h (Abstract base class) and Subclasses

```
class Consumer {
protected:
    int ConsumerID;
    string name;
    float unitConsumed;
public:
    Consumer();
    Consumer(int id, string n, float unitC);
    virtual ~Consumer();
    virtual float calculateBill() = 0;
    int getID();
    string getName();
    float getunitConsumed();
    virtual string Display();
    virtual string saveData();
    virtual void updateUnits(float units, float peak, float offPeak, float extra) = 0;
};
class ResidentialConsumer : public Consumer { ... };
class CommercialConsumer : public Consumer { ... };
class SolarConsumer : public ResidentialConsumer { ... };
```

- **Responsibilities of Subclasses:**
 - **ResidentialConsumer:** Stores and calculates bills with peak/off-peak units.

- CommercialConsumer: Computes commercial rate bills.
- SolarConsumer: Inherits from ResidentialConsumer, adds solar exported logic (net-metering).

Admin.h

```
class Admin {
protected:
    Consumer* consumer[100];
    int count;
public:
    static float residentialPeakRate, residentialOffPeakRate, commercialRate,
solarRate;
    // Methods for: adding/searching/updating consumers, managing rates, statistics,
and registering users.
};
```

- **Role:** Provides all methods for consumer management, statistics, and rate-setting, helping bridge between business rules and data persistence.

FileManager.h

```
class FileManager {
public:
    static void saveRates();
    static void loadRates();
    static void saveConsumers(Consumer* consumer[], int count);
    static void loadConsumers(Consumer* consumer[], int& count);
    // ... many more helpers for users, bills, I/O ...
};
```

- **Role:** Handles all low-level file I/O, keeping the saving and loading of rates, users, consumers, and bills unified. All business logic relies on this class for data persistence.

OOP Concepts Used

Inheritance

- ResidentialConsumer, CommercialConsumer, and SolarConsumer (which extends ResidentialConsumer) all inherit from the base Consumer class.

Encapsulation

- All essential class data is protected or private. State is only accessed/mutated via class methods, not directly.

Polymorphism

- Core business and storage methods operate on base class pointers (Consumer*), relying on virtual methods for true object behavior.
- The system leverages C++'s virtual/override for calculateBill, saveData, updateUnits, etc.
- Uses dynamic casting (e.g., check for SolarConsumer*) for logic branching.

Abstraction

- The user interface is cleanly separated from business logic and persistence logic.
- All file operations are abstracted into FileManager, and UI code never touches disk or business logic directly.

Data Storage Approach

- **File Handling Used:**
 - All users, consumer, and bill data is saved/loaded from text files via C++ file I/O and optionally QFile/QTextStream (Qt Classes for File Handling).

Main Code Files & Descriptions

- **main.cpp (Project):**
Entry point for the console-based version; manages initial menus (For Terminal Based Output) and core program execution.
- **main.cpp (QtMultiWindow):**
Starts the GUI application and opens the initial login window in Qt.
- **AdminDashboard.cpp (QtMultiWindow):**
Implements the admin interface for managing consumers, rates, statistics, and users via a windowed GUI.
- **UserDashboard.cpp (QtMultiWindow):**
Implements the user interface for customers to view and pay bills and see their billing history.
- **LoginWindow.cpp (QtMultiWindow):**
Provides the login window logic, handling sign-in for admin and users.

- **SystemContext.cpp (QtMultiWindow):**
Central class implementing all core logic for authentication, billing, data management, and business rules for the GUI.
- **Consumer.h / Consumer.cpp:**
Defines the abstract base class and subclasses for consumer types, encapsulating all billing logic and properties.
- **Admin.h / Admin.cpp:**
Provides methods for admin operations: adding/searching/updating/displaying consumers, rate-setting, statistics, and user registration.
- **FileManager.h / FileManager.cpp:**
Handles all file I/O for loading and saving consumers, users, rates, and bills, centralizing persistence logic.
- **LoginManager.h / LoginManager.cpp:**
Defines logic for managing user accounts and credentials.
- **MonthlyRecords.h / MonthlyRecords.cpp:**
Manages the structure and storage of monthly billing records for consumers.
- **Input.h / Input.cpp:**
Contains input validation and helper functions for the console-based workflow.

Conclusion:

This project integrates robust object-oriented programming principles with user-friendly interfaces and reliable data persistence methods. Its modular design enables easy maintenance and extension, while clear separation between UI, business logic, and data layers demonstrates sound software engineering. It provides a practical, complete solution for modern electricity billing and management needs, adaptable for both academic and real-world deployments.

UML Diagram :

- **Billing Hierarchy:** Abstract base class Consumer splits into CommercialConsumer and ResidentialConsumer, with a SolarConsumer extension for net-metering.
- **Control & Logic:** Admin handles rates and consumer data, while SystemContext/QtSystemController pass formatted data (QString) to the GUI.
- **Data & Access:** MonthlyRecords tracks payments, LoginManager secures credentials, and FileManager isolates file I/O for persistence.

